

Docker Image uploaded on Docker Hub

 Explore My Hub

Search Docker Hub

Using 0 of 1 private repositories.

 itachiwastaken
Docker Personal

 Repositories

 Hardened Images

 Collaborations

 Settings

Default privacy

Notifications

 Billing

 Usage

Pulls

Storage

[Repositories](#) / [fogisimage](#) / [General](#)

itachiwastaken/fogisimage

Last pushed 8 minutes ago · Repository size: 259.8 MB · ☆0 · ↓17

[Add a description](#)

[Add a category](#)

[Public view](#)

Docker commands

To push a new tag to this repository:

```
docker push itachiwastaken/fogisimage:tagname
```

[General](#) Tags Image Management Collaborators Webhooks Settings

Tags

DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	8 minutes

[See all](#)



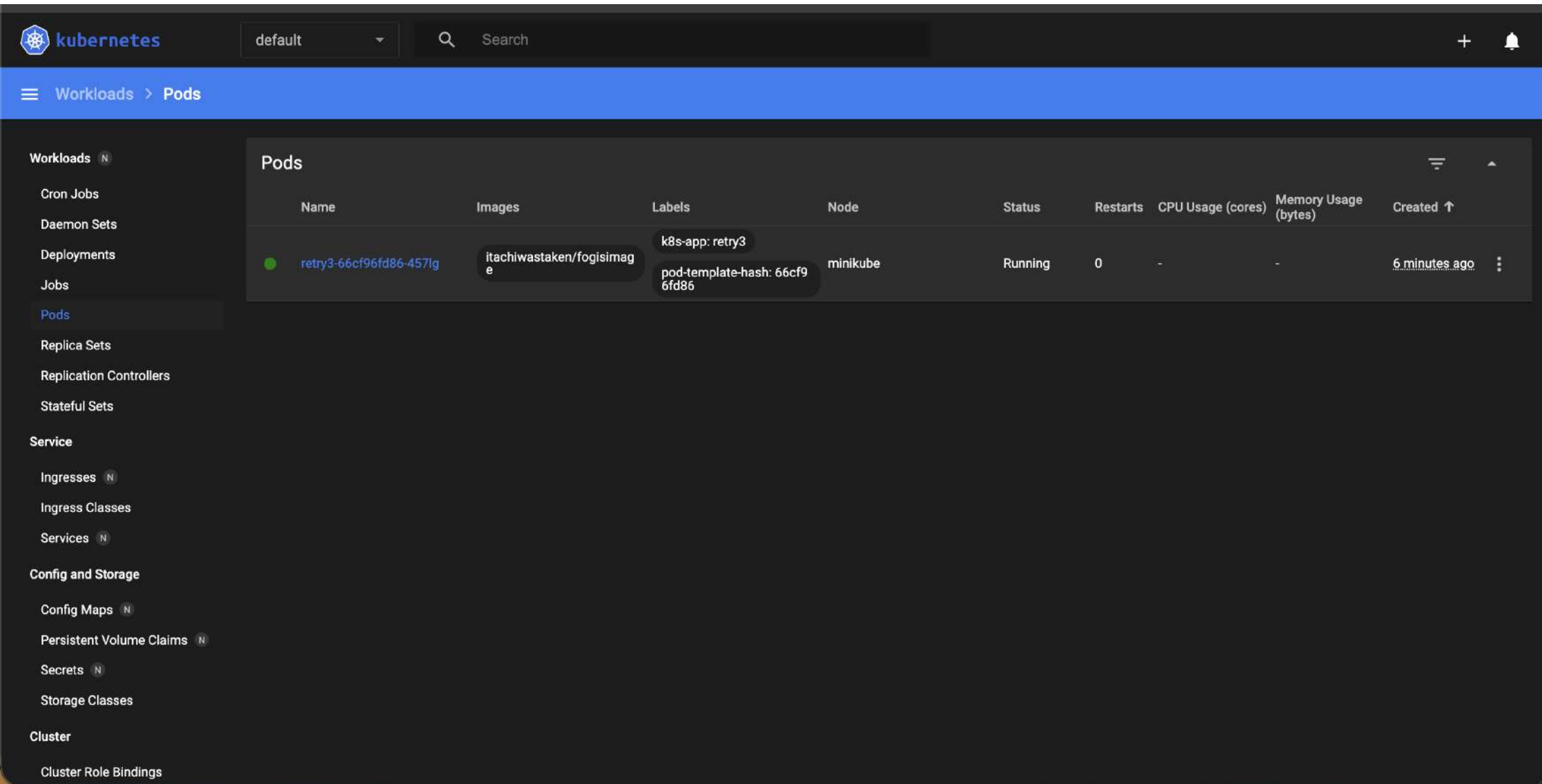
Build with
Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data

Deploy/Running the service in a Pod



The image is a screenshot of the Kubernetes Dashboard. At the top, there's a header with the Kubernetes logo, a 'default' namespace dropdown, a search bar, and a notification bell. Below the header is a blue navigation bar with 'Workloads > Pods' selected. On the left is a sidebar menu with categories: Workloads (Cron Jobs, Daemon Sets, Deployments, Jobs, Pods, Replica Sets, Replication Controllers, Stateful Sets), Service (Ingresses, Ingress Classes, Services), Config and Storage (Config Maps, Persistent Volume Claims, Secrets, Storage Classes), and Cluster (Cluster Role Bindings). The main area is titled 'Pods' and contains a table with one pod. The pod is named 'retry3-66cf96fd86-457lg', uses the 'itachiwastaken/fogisimage' image, has labels 'k8s-app: retry3' and 'pod-template-hash: 66cf96fd86', is running on the 'minikube' node, and has a status of 'Running' with 0 restarts. It was created 6 minutes ago.

Kubernetes

default

Search

Workloads > Pods

Workloads

- Cron Jobs
- Daemon Sets
- Deployments
- Jobs
- Pods**
- Replica Sets
- Replication Controllers
- Stateful Sets

Service

- Ingresses
- Ingress Classes
- Services

Config and Storage

- Config Maps
- Persistent Volume Claims
- Secrets
- Storage Classes

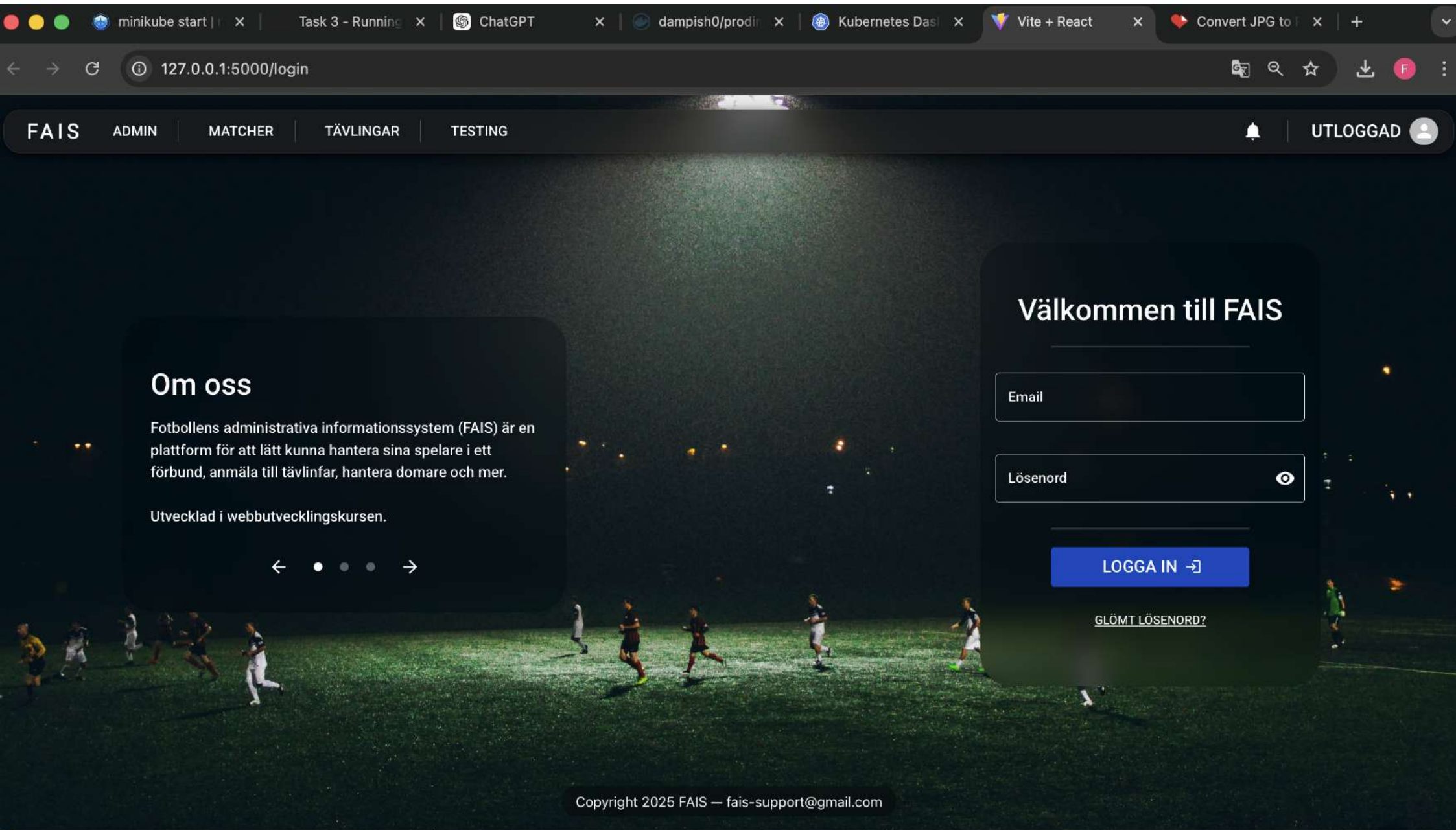
Cluster

- Cluster Role Bindings

Pods

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created ↑
 retry3-66cf96fd86-457lg	itachiwastaken/fogisimage	k8s-app: retry3 pod-template-hash: 66cf96fd86	minikube	Running	0	-	-	6 minutes ago

Verifying that the Service is running



Logs from the running service

 **kubernetes**

default

Search

+



Workloads > Pods > retry3-66cf96fd86-457lg > Logs

Workloads

Cron Jobs

Daemon Sets

Deployments

Jobs

Pods

Replica Sets

Replication Controllers

Stateful Sets

Service

Ingresses

Ingress Classes

Services

Config and Storage

Config Maps

Persistent Volume Claims

Secrets

Logs from **retry3** in **retry3-66cf96f...**

```
[dotenv@17.3.1] injecting env (0) from .env -- tip: 🛡️ prevent committing .env to code: https://dotenvx.com/precommit
[dotenv@17.3.1] injecting env (0) from .env -- tip: ⚙️ specify custom .env file path with { path: '/custom/path/.env' }
[dotenv@17.3.1] injecting env (0) from .env -- tip: ⚙️ override existing env vars with { override: true }
[dotenv@17.3.1] injecting env (0) from .env -- tip: 🔒 encrypt with Dotenvx: https://dotenvx.com
MONGO CONNECTED
Server at port: 5000
req method is GET, Req url is /
req method is GET, Req url is /assets/index-BGV7QwYa.js
req method is GET, Req url is /assets/index-DevYWjwo.css
req method is GET, Req url is /assets/Olsen-CzRLVNeD.png
req method is GET, Req url is /assets/Mbappe-C2cWgo6n.png
req method is GET, Req url is /assets/Noah-BV5ZukS6.png
req method is GET, Req url is /api/auth/check-auth
req method is GET, Req url is /assets/roboto-latin-500-normal-CkrA1NAy.woff2
req method is GET, Req url is /assets/Malmo-De91a9rn.png
req method is GET, Req url is /vite.svg
req method is GET, Req url is /assets/frontpagebg-QfYffJw2.jpg
req method is GET, Req url is /vite.svg
req method is GET, Req url is /api/auth/check-auth
req method is GET, Req url is /login
req method is GET, Req url is /assets/index-BGV7QwYa.js
req method is GET, Req url is /assets/index-DevYWjwo.css
req method is GET, Req url is /assets/frontpagebg-QfYffJw2.jpg
req method is GET, Req url is /assets/roboto-latin-500-normal-CkrA1NAy.woff2
req method is GET, Req url is /api/auth/check-auth
req method is GET, Req url is /vite.svg
req method is POST, Req url is /api/auth/login
```

Logs from Feb 25, 2026 to Feb 25, 2026 UTC

The Load balancer for our service

The screenshot shows the Kubernetes dashboard interface. At the top, there's a header with the Kubernetes logo, a namespace dropdown set to 'default', and a search bar. Below the header, a blue navigation bar shows 'Service > Services'. On the left, a sidebar lists various Kubernetes resources: Workloads (Cron Jobs, Daemon Sets, Deployments, Jobs, Pods, Replica Sets, Replication Controllers, Stateful Sets), Service (Ingresses, Ingress Classes, Services), and Config and Storage (Config Maps, Persistent Volume Claims, Secrets, Storage Classes). The 'Services' section is highlighted. The main content area displays a table of services. The first service, 'retry3', is highlighted with a green dot and has its 'Type' set to 'LoadBalancer', which is circled in red. The second service, 'kubernetes', is also highlighted with a green dot and has its 'Type' set to 'ClusterIP'. The table columns are Name, Labels, Type, Cluster IP, Internal Endpoints, and External Endpoints.

Name	Labels	Type	Cluster IP	Internal Endpoints	External Endpoints
retry3	k8s-app: retry3	LoadBalancer	10.98.143.61	retry3:5000 TCP retry3:31786 TCP	127.0.0.1:5000 ↗
kubernetes	component: apiserver provider: kubernetes	ClusterIP	10.96.0.1	kubernetes:443 TCP kubernetes:0 TCP	-